Python Interview Coding Questions & Solutions

Built-in and manual solutions • Python, SQL, backend, full-stack, and data engineering

Preparation guide for an engineer with ~4 years of experience

How to use this guide

- Attempt each question before reading the solution.
- Built-in solutions use Python language features or standard-library tools. Manual solutions implement the core algorithm directly.
- Explain assumptions, edge cases, time complexity, and space complexity during interviews.
- The SQL examples assume PostgreSQL and use illustrative table/column names.
- Production examples are starting points; adapt validation, security, persistence, and error handling to the real application.

Contents

- Part 1: Strings
- Part 2: Lists and arrays
- Part 3: Dictionaries, sets, and logic
- Part 4: Decorators and Python internals
- Part 5: SQL questions
- Part 6: FastAPI, testing, retries, manufacturing data, React, and Databricks

Part 1 — Strings

1. Reverse a string

Question: Return a string in reverse order.

Approach A — Built-in / standard library:

```
s = "hello"
print(s[::-1])
```

Approach B — Manual implementation:

```
def reverse_string(s):
    chars = []
    for ch in s:
        chars.append(ch)
    result = ""
    for ch in chars:
        result = ch + result
    return result

print(reverse_string("hello"))
```

Complexity: Slicing is concise. Manual repeated string concatenation can be $O(n^2)$; using a list and reversed iteration can keep it $O(n)$.

2. Check palindrome

Question: Check whether text reads the same forwards and backwards.

Approach A — Built-in / standard library:

```
def palindrome(s):
    cleaned = ''.join(ch.lower() for ch in s if ch.isalnum())
    return cleaned == cleaned[::-1]
```

```
print(palindrome("Never odd or even"))
```

Approach B — Manual implementation:

```
def palindrome(s):
    left, right = 0, len(s) - 1
    while left < right:
        if s[left].lower() != s[right].lower():
            return False
        left += 1
        right -= 1
    return True

print(palindrome("level"))
```

Complexity: O(n) time. The manual version compares characters exactly apart from case; add skip logic if spaces/punctuation should be ignored.

3. First non-repeating character

Question: Return the first character with frequency one, or None.

Approach A — Built-in / standard library:

```
from collections import Counter
def first_unique(s):
    counts = Counter(s)
    return next((ch for ch in s if counts[ch] == 1), None)
print(first_unique("swiss"))  # w
```

Approach B — Manual implementation:

```
def first_unique(s):
    counts = {}
    for ch in s:
        counts[ch] = counts.get(ch, 0) + 1
    for ch in s:
        if counts[ch] == 1:
            return ch
    return None
print(first_unique("swiss"))
```

Complexity: O(n) average time, O(k) space where k is the number of distinct characters.

4. Second non-repeating character

Question: Return the second unique character in original order.

Approach A — Built-in / standard library:

```
from collections import Counter
def second_unique(s):
    counts = Counter(s)
    unique = [ch for ch in s if counts[ch] == 1]
    return unique[1] if len(unique) > 1 else None
print(second_unique("aabbcdef"))  # d
```

Approach B — Manual implementation:

```
def second_unique(s):
    counts = {}
    for ch in s:
        counts[ch] = counts.get(ch, 0) + 1
    seen_unique = 0
    for ch in s:
        if counts[ch] == 1:
            seen_unique += 1
            if seen_unique == 2:
                return ch
    return None
print(second_unique("aabbcdef"))
```

Complexity: O(n) time, O(k) space.

5. Count character frequencies

Question: Count each character in a string.

Approach A — Built-in / standard library:

```
from collections import Counter
print(Counter("banana"))
```

Approach B — Manual implementation:

```
def char_counts(s):
    counts = {}
    for ch in s:
        if ch not in counts:
            counts[ch] = 0
        counts[ch] += 1
    return counts
print(char_counts("banana"))
```

Complexity: O(n) average time, O(k) space.

6. Check anagrams

Question: Check whether two strings have identical character frequencies.

Approach A — Built-in / standard library:

```
from collections import Counter
def anagram(a, b):
    return Counter(a) == Counter(b)
print(anagram("listen", "silent"))
```

Approach B — Manual implementation:

```
def anagram(a, b):
    if len(a) != len(b):
        return False
    counts = {}
    for ch in a:
        counts[ch] = counts.get(ch, 0) + 1
    for ch in b:
        if ch not in counts:
            return False
        counts[ch] -= 1
        if counts[ch] < 0:
            return False
    return True
print(anagram("listen", "silent"))
```

Complexity: O(n) average time, O(k) space. Case and whitespace are significant unless normalized.

7. Find duplicate characters

Question: Return repeated characters once, in first-seen order.

Approach A — Built-in / standard library:

```
from collections import Counter
def duplicate_chars(s):
    counts = Counter(s)
    return list(dict.fromkeys(ch for ch in s if counts[ch] > 1))
print(duplicate_chars("programming"))
```

Approach B — Manual implementation:

```
def duplicate_chars(s):
    counts = {}
    for ch in s:
        counts[ch] = counts.get(ch, 0) + 1
    result, added = [], set()
    for ch in s:
        if counts[ch] > 1 and ch not in added:
            result.append(ch)
            added.add(ch)
    return result
print(duplicate_chars("programming"))
```

Complexity: O(n) average time, O(k) space.

8. Longest substring without repeated characters

Question: Return the length of the longest unique-character substring.

Approach A — Built-in / standard library:

```
def longest_unique(s):
    last = {}
    start = best = 0
    for i, ch in enumerate(s):
        if ch in last and last[ch] >= start:
            start = last[ch] + 1
        last[ch] = i
        best = max(best, i - start + 1)
    return best
print(longest_unique("abcabcbb"))  # 3
```

Approach B — Manual implementation:

```
def longest_unique_manual(s):
    best = 0
    for start in range(len(s)):
        seen = set()
        for end in range(start, len(s)):
            if s[end] in seen:
                break
            seen.add(s[end])
            best = max(best, end - start + 1)
    return best
print(longest_unique_manual("abcabcbb"))
```

Complexity: Sliding-window solution O(n) average; manual nested scan $O(n^2)$ worst case.

Part 2 — Lists and arrays

9. Find duplicate list elements

Question: Return each duplicated value once, preserving order of repeated discovery.

Approach A — Built-in / standard library:

```
def duplicates(nums):
    return list(dict.fromkeys(x for x in nums if nums.count(x) > 1))
print(duplicates([1, 2, 3, 2, 1]))
```

Approach B — Manual implementation:

```
def duplicates(nums):
    seen, added, result = set(), set(), []
    for x in nums:
        if x in seen and x not in added:
            result.append(x)
            added.add(x)
        seen.add(x)
    return result
print(duplicates([1, 2, 3, 2, 1]))
```

Complexity: Built-in count() version $O(n^2)$; set-based manual solution O(n) average. Prefer the set-based solution in interviews.

10. Count occurrences of numbers

Question: Map each number to its frequency.

Approach A — Built-in / standard library:

```
from collections import Counter
print(Counter([1, 2, 3, 2, 1, 2]))
```

Approach B — Manual implementation:

```
def counts(nums):
    result = {}
    for x in nums:
        result[x] = result.get(x, 0) + 1
```

```
    return result
print(counts([1, 2, 3, 2, 1, 2]))
```

Complexity: O(n) average time, O(k) space.

11. Second-largest distinct number

Question: Return second-largest distinct value or None.

Approach A — Built-in / standard library:

```
def second_largest(nums):
    values = sorted(set(nums))
    return values[-2] if len(values) >= 2 else None
print(second_largest([10, 5, 8, 10, 7]))  # 8
```

Approach B — Manual implementation:

```
def second_largest(nums):
    largest = second = None
    for x in nums:
        if largest is None or x > largest:
            if x != largest:
                second, largest = largest, x
        elif x != largest and (second is None or x > second):
            second = x
    return second
print(second_largest([10, 5, 8, 10, 7]))
```

Complexity: Sort approach O(n log n); one-pass approach O(n) time and O(1) extra space.

12. Find missing numbers in 1..n

Question: Return values absent from the list.

Approach A — Built-in / standard library:

```
def missing(nums, n):
    return sorted(set(range(1, n + 1)) - set(nums))
print(missing([1, 2, 4, 6], 6))  # [3, 5]
```

Approach B — Manual implementation:

```
def missing(nums, n):
    present = [False] * (n + 1)
    for x in nums:
        if 1 <= x <= n:
            present[x] = True
    return [x for x in range(1, n + 1) if not present[x]]
print(missing([1, 2, 4, 6], 6))
```

Complexity: O(n) time and O(n) space.

13. Remove duplicates preserving order

Question: Keep only the first occurrence of each item.

Approach A — Built-in / standard library:

```
def unique_order(nums):
    return list(dict.fromkeys(nums))
print(unique_order([3, 1, 3, 2, 1]))
```

Approach B — Manual implementation:

```
def unique_order(nums):
    seen, result = set(), []
    for x in nums:
        if x not in seen:
            seen.add(x)
            result.append(x)
    return result
print(unique_order([3, 1, 3, 2, 1]))
```

Complexity: O(n) average time, O(k) space.

14. Two Sum

Question: Return indices of two values that add to target.

Approach A — Built-in / standard library:

```
def two_sum(nums, target):
    seen = {}
    for i, x in enumerate(nums):
        if target - x in seen:
            return [seen[target - x], i]
        seen[x] = i
    return None
print(two_sum([2, 7, 11, 15], 9))
```

Approach B — Manual implementation:

```
def two_sum_brute(nums, target):
    for i in range(len(nums)):
        for j in range(i + 1, len(nums)):
            if nums[i] + nums[j] == target:
                return [i, j]
    return None
print(two_sum_brute([2, 7, 11, 15], 9))
```

Complexity: Hash map O(n) average time/O(n) space; brute force $O(n^2)$ time.

15. Move zeros to the end

Question: Preserve nonzero order and move all zeros after them.

Approach A — Built-in / standard library:

```
def move_zeros(nums):
    nonzero = [x for x in nums if x != 0]
    return nonzero + [0] * (len(nums) - len(nonzero))
print(move_zeros([0, 1, 0, 3, 12]))
```

Approach B — Manual implementation:

```
def move_zeros(nums):
    write = 0
    for x in nums:
        if x != 0:
            nums[write] = x
            write += 1
    while write < len(nums):
        nums[write] = 0
        write += 1
    return nums
print(move_zeros([0, 1, 0, 3, 12]))
```

Complexity: O(n) time. In-place manual approach uses O(1) extra space.

16. Common elements in two lists

Question: Return unique values present in both lists.

Approach A — Built-in / standard library:

```
def common(a, b):
    return list(set(a) & set(b))
print(common([1, 2, 2, 3], [2, 3, 4]))
```

Approach B — Manual implementation:

```
def common_manual(a, b):
    bset = set(b)
    result, seen = [], set()
    for x in a:
        if x in bset and x not in seen:
            result.append(x)
            seen.add(x)
    return result
print(common_manual([1, 2, 2, 3], [2, 3, 4]))
```

Complexity: O(n+m) average time using sets. Set result order is not guaranteed.

17. Merge two sorted lists

Question: Merge without sorting the combined list.

Approach A — Built-in / standard library:

```
def merge_sorted(a, b):
    return sorted(a + b)
print(merge_sorted([1, 4, 7], [2, 3, 8]))
```

Approach B — Manual implementation:

```
def merge_sorted_manual(a, b):
    i = j = 0
    out = []
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            out.append(a[i]); i += 1
        else:
            out.append(b[j]); j += 1
    out.extend(a[i:])
    out.extend(b[j:])
    return out
print(merge_sorted_manual([1, 4, 7], [2, 3, 8]))
```

Complexity: Manual merge O(n+m); sort approach O((n+m) log(n+m)).

18. Maximum subarray sum

Question: Find the maximum sum of any non-empty contiguous subarray.

Approach A — Built-in / standard library:

```
def max_subarray(nums):
    best = current = nums[0]
    for x in nums[1:]:
        current = max(x, current + x)
        best = max(best, current)
    return best
print(max_subarray([-2,1,-3,4,-1,2,1,-5,4]))  # 6
```

Approach B — Manual implementation:

```
def max_subarray_brute(nums):
    best = nums[0]
    for i in range(len(nums)):
        total = 0
        for j in range(i, len(nums)):
            total += nums[j]
            best = max(best, total)
    return best
print(max_subarray_brute([-2,1,-3,4,-1,2,1,-5,4]))
```

Complexity: Kadane O(n) time/O(1) space; brute force O(n²) time.

Part 3 — Dictionaries, sets, and logic

19. First duplicate element

Question: Return the first value encountered for a second time.

Approach A — Built-in / standard library:

```
def first_duplicate(nums):
    seen = set()
    for x in nums:
        if x in seen:
            return x
        seen.add(x)
    return None
print(first_duplicate([4, 2, 7, 2, 4]))  # 2
```

Approach B — Manual implementation:

```
def first_duplicate_brute(nums):
    for j in range(len(nums)):
```

```
        for i in range(j):
            if nums[i] == nums[j]:
                return nums[j]
    return None
print(first_duplicate_brute([4, 2, 7, 2, 4]))
```

Complexity: Set approach O(n) average; brute force O(n²).

20. Most frequent element

Question: Return the most frequent value; None for an empty list.

Approach A — Built-in / standard library:

```
from collections import Counter
def most_frequent(nums):
    if not nums: return None
    counts = Counter(nums)
    return max(counts, key=counts.get)
print(most_frequent([1, 3, 3, 2, 1, 3]))
```

Approach B — Manual implementation:

```
def most_frequent_manual(nums):
    counts = {}
    for x in nums:
        counts[x] = counts.get(x, 0) + 1
    best, best_count = None, 0
    for x in nums:
        if counts[x] > best_count:
            best, best_count = x, counts[x]
    return best
print(most_frequent_manual([1, 3, 3, 2, 1, 3]))
```

Complexity: O(n) average time, O(k) space.

21. Group anagrams

Question: Group words with the same character counts.

Approach A — Built-in / standard library:

```
from collections import defaultdict
def group_anagrams(words):
    groups = defaultdict(list)
    for word in words:
        groups[''.join(sorted(word))].append(word)
    return list(groups.values())
print(group_anagrams(["eat","tea","tan","ate","nat","bat"]))
```

Approach B — Manual implementation:

```
def group_anagrams_manual(words):
    groups = {}
    for word in words:
        key = [0] * 26
        for ch in word.lower():
            if 'a' <= ch <= 'z':
                key[ord(ch) - ord('a')] += 1
        key = tuple(key)
        groups.setdefault(key, []).append(word)
    return list(groups.values())
print(group_anagrams_manual(["eat","tea","tan","ate","nat","bat"]))
```

Complexity: Sorting key O(w·L log L); frequency key O(w·L) for lowercase English letters.

Note: Manual frequency key assumes a-z characters; adapt for other alphabets.

22. Common keys in dictionaries

Question: Return keys present in both dictionaries.

Approach A — Built-in / standard library:

```
def common_keys(a, b):
    return a.keys() & b.keys()
print(common_keys({"x":1,"y":2}, {"y":9,"z":3}))
```

Approach B — Manual implementation:

```
def common_keys_manual(a, b):
    result = []
    for key in a:
        if key in b:
            result.append(key)
    return result
print(common_keys_manual({"x":1,"y":2}, {"y":9,"z":3}))
```

Complexity: O(n) average time for the manual loop.

23. Flatten a nested list

Question: Flatten arbitrarily nested lists such as [1, [2, [3]], 4].

Approach A — Built-in / standard library:

```
def flatten(items):
    out = []
    for item in items:
        if isinstance(item, list):
            out.extend(flatten(item))
        else:
            out.append(item)
    return out
print(flatten([1, [2, [3]], 4]))
```

Approach B — Manual implementation:

```
def flatten_stack(items):
    out, stack = [], list(reversed(items))
    while stack:
        item = stack.pop()
        if isinstance(item, list):
            stack.extend(reversed(item))
        else:
            out.append(item)
    return out
print(flatten_stack([1, [2, [3]], 4]))
```

Complexity: O(n) time and O(n) space for n total elements.

24. Sort dictionary by values

Question: Return a dictionary ordered by values ascending.

Approach A — Built-in / standard library:

```
data = {"a": 3, "b": 1, "c": 2}
print(dict(sorted(data.items(), key=lambda pair: pair[1])))
```

Approach B — Manual implementation:

```
def sort_values_manual(data):
    items = list(data.items())
    for i in range(len(items)):
        for j in range(len(items)-1-i):
            if items[j][1] > items[j+1][1]:
                items[j], items[j+1] = items[j+1], items[j]
    return dict(items)
print(sort_values_manual({"a":3, "b":1, "c":2}))
```

Complexity: Built-in sort O(n log n); manual bubble sort O(n^2).

25. Intersection of multiple lists

Question: Find values present in every list.

Approach A — Built-in / standard library:

```
def intersection_all(lists):
    if not lists: return set()
    return set.intersection(*(set(x) for x in lists))
print(intersection_all([[1,2,3],[2,3,4],[0,2,3]]))
```

Approach B — Manual implementation:

```
def intersection_all_manual(lists):
    if not lists: return []
    result = []
    for x in lists[0]:
        if x not in result:
            if all(x in group for group in lists[1:]):
                result.append(x)
    return result
print(intersection_all_manual([[1,2,3],[2,3,4],[0,2,3]]))
```

Complexity: Set approach typically O(total input size); manual list membership can be slower.

Part 4 — Decorators and Python internals

26. Execution-time decorator

Question: Measure and print how long a function takes.

Approach A — Built-in / standard library:

```
import time
from functools import wraps
def measure_time(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        try:
            return func(*args, **kwargs)
        finally:
            print(f"{func.__name__}: {time.perf_counter()-start:.4f}s")
    return wrapper

@measure_time
def work():
    return sum(range(100000))
print(work())
```

Approach B — Manual implementation:

```
import time
def measure_time(func):
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        print(func.__name__, time.perf_counter() - start)
        return result
    return wrapper

@measure_time
def work():
    return sum(range(100000))
print(work())
```

Complexity: Use functools.wraps to preserve name/docstring. The finally version logs even if the function raises.

27. Retry decorator with arguments

Question: Retry transient failures with exponential backoff.

Approach A — Built-in / standard library:

```
import time
from functools import wraps
def retry(attempts=3, base_delay=0.5,
          retryable=(TimeoutError, ConnectionError)):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(attempts):
                try:
                    return func(*args, **kwargs)
                except retryable:
                    if attempt == attempts - 1:
                        raise
                    time.sleep(base_delay * (2 ** attempt))
        return wrapper
    return decorator
```

```
@retry(attempts=3)
def fetch():
    return {"ok": True}
print(fetch())
```

Approach B — Manual implementation:

```
import time
def retry_call(func, attempts=3, delay=1):
    for attempt in range(attempts):
        try:
            return func()
        except (TimeoutError, ConnectionError):
            if attempt == attempts - 1:
                raise
            time.sleep(delay)
    return None

print(retry_call(lambda: "success"))
```

Complexity: Retry only transient errors. Consider jitter, timeout budgets, Retry-After, and idempotency in production.

28. Decorator that accepts its own arguments

Question: Repeat a decorated function n times using @repeat(n).

Approach A — Built-in / standard library:

```
from functools import wraps
def repeat(n):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            result = None
            for _ in range(n):
                result = func(*args, **kwargs)
            return result
        return wrapper
    return decorator

@repeat(3)
def greet():
    print("Hello")
greet()
```

Approach B — Manual implementation:

```
def repeat(n):
    def decorator(func):
        def wrapper(*args, **kwargs):
            result = None
            i = 0
            while i < n:
                result = func(*args, **kwargs)
                i += 1
            return result
        return wrapper
    return decorator

@repeat(3)
def greet():
    print("Hello")
greet()
```

Complexity: The decorator has three levels: configuration, function, wrapper.

29. Generator using yield

Question: Generate squares lazily.

Approach A — Built-in / standard library:

```
def squares(n):
    for x in range(n):
        yield x*x
print(list(squares(5)))
```

Approach B — Manual implementation:

```
class Squares:
    def __init__(self, n):
        self.n, self.i = n, 0
    def __iter__(self):
        return self
    def __next__(self):
        if self.i >= self.n:
            raise StopIteration
        value = self.i * self.i
        self.i += 1
        return value
print(list(Squares(5)))
```

Complexity: O(n) time to consume all results; generator state is O(1).

30. Shallow copy vs deep copy

Question: Explain the difference and demonstrate it.

```
import copy
original = [[1, 2], [3, 4]]
shallow = copy.copy(original)
deep = copy.deepcopy(original)
original[0].append(99)
print(shallow)  # nested change is shared
print(deep)     # nested list remains independent
```

A shallow copy duplicates the outer container but shares nested objects. A deep copy recursively copies nested objects where supported.

31. Mutable vs immutable

```
a = [1, 2]
b = a
b.append(3)
print(a)  # [1, 2, 3]

x = "hi"
y = x
y += "!"
print(x, y)  # hi hi!
```

Lists, dicts, and sets are mutable. Integers, strings, and booleans are immutable. Tuples are immutable containers, but can contain mutable objects.

32. Exception handling

```
try:
    value = int("123")
except ValueError:
    print("Invalid number")
else:
    print("Parsed:", value)
finally:
    print("Cleanup runs regardless")
```

Use finally for cleanup; use context managers for resources such as files and connections.

33. List comprehension vs generator expression

```
squares_list = [x*x for x in range(1_000_000)]  # all values stored
squares_gen = (x*x for x in range(1_000_000))       # lazy iteration
```

A list stores all results; a generator expression computes values as consumed.

34. Context manager

```
from contextlib import contextmanager
@contextmanager
def resource():
    print("Acquire")
    try:
        yield "resource"
    finally:
        print("Release")

with resource() as value:
    print("Use", value)
```

A context manager structures setup and cleanup around a with block.

35. Threading vs multiprocessing vs asyncio

- Threading: useful for I/O-bound tasks; CPython's GIL limits parallel execution of CPU-bound Python bytecode.
- Multiprocessing: separate processes can execute CPU-bound work in parallel, with extra memory and communication costs.
- asyncio: cooperative concurrency for I/O when libraries support async; it does not automatically parallelize CPU-heavy work.

Part 5 — SQL interview questions (PostgreSQL examples)

Assumed tables: employees(employee_id, name, department_id, salary), departments(department_id, department_name), customers(customer_id, name), orders(order_id, customer_id, order_date, amount).

36. Second-highest distinct salary

Question: Second-highest distinct salary.

Solution:

```
SELECT MAX(salary) AS second_highest
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

Alternative / notes:

```
SELECT salary
FROM (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
  FROM employees
) s
WHERE rnk = 2
LIMIT 1;
```

DENSE_RANK handles ties as distinct salary levels.

37. Find duplicate values

Question: Find duplicate values.

Solution:

```
SELECT name, COUNT(*) AS occurrences
FROM employees
GROUP BY name
HAVING COUNT(*) > 1;
```

Alternative / notes:

```
-- Replace name with the relevant business key, e.g. email.
SELECT email, COUNT(*)
FROM employees
GROUP BY email
HAVING COUNT(*) > 1;
```

GROUP BY creates groups; HAVING filters aggregate results.

38. Employees earning above department average

Question: Employees earning above department average.

Solution:

```
SELECT e.*
FROM employees e
JOIN (
  SELECT department_id, AVG(salary) AS avg_salary
  FROM employees
  GROUP BY department_id
) d ON d.department_id = e.department_id
WHERE e.salary > d.avg_salary;
```

Alternative / notes:

```
SELECT *
FROM (
  SELECT e.*,
         AVG(salary) OVER (PARTITION BY department_id) AS dept_avg
  FROM employees e
) x
WHERE salary > dept_avg;
```

The second version uses a window function.

39. Top 3 salary levels per department

Question: Top 3 salary levels per department.

Solution:

```
SELECT *
FROM (
  SELECT e.*,
         DENSE_RANK() OVER (
           PARTITION BY department_id ORDER BY salary DESC
         ) AS rnk
  FROM employees e
) x
WHERE rnk <= 3;
```

Alternative / notes:

```
-- Use ROW_NUMBER() instead of DENSE_RANK() if you need at most 3 rows per department.
```

DENSE_RANK includes ties; ROW_NUMBER assigns unique row positions.

40. Customers with no orders

Question: Customers with no orders.

Solution:

```
SELECT c.*
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.order_id IS NULL;
```

Alternative / notes:

```
SELECT c.*
FROM customers c
WHERE NOT EXISTS (
  SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id
);
```

This is an anti-join pattern.

41. Running total

Question: Running total.

Solution:

```
SELECT order_id, customer_id, order_date, amount,
       SUM(amount) OVER (
         PARTITION BY customer_id
         ORDER BY order_date, order_id
         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS running_total
FROM orders;
```

Alternative / notes:

```
-- Keep a deterministic tie-breaker in ORDER BY when dates can match.
```

Window aggregate computes a cumulative sum.

42. Latest order per customer

Question: Latest order per customer.

Solution:

```
SELECT *
FROM (
  SELECT o.*,
         ROW_NUMBER() OVER (
           PARTITION BY customer_id
           ORDER BY order_date DESC, order_id DESC
         ) AS rn
  FROM orders o
) x
WHERE rn = 1;
```

Alternative / notes:

```
-- PostgreSQL alternative:
SELECT DISTINCT ON (customer_id) *
FROM orders
ORDER BY customer_id, order_date DESC, order_id DESC;
```

DISTINCT ON is PostgreSQL-specific.

43. Month-over-month revenue growth

Question: Month-over-month revenue growth.

Solution:

```
WITH monthly AS (
  SELECT date_trunc('month', order_date) AS month,
         SUM(amount) AS revenue
  FROM orders
  GROUP BY 1
), compared AS (
  SELECT month, revenue,
         LAG(revenue) OVER (ORDER BY month) AS previous_revenue
  FROM monthly
)
SELECT month, revenue, previous_revenue,
       CASE WHEN previous_revenue IS NULL OR previous_revenue = 0 THEN NULL
            ELSE 100.0 * (revenue - previous_revenue) / previous_revenue
       END AS growth_pct
FROM compared
ORDER BY month;
```

Alternative / notes:

```
-- If missing calendar months matter, build a complete month series first.
```

Uses date_trunc and LAG in PostgreSQL.

44. Delete duplicate records, keeping one

Question: Delete duplicate records, keeping one.

Solution:

```
WITH ranked AS (
  SELECT employee_id,
         ROW_NUMBER() OVER (
           PARTITION BY name ORDER BY employee_id
         ) AS rn
  FROM employees
)
DELETE FROM employees e
USING ranked r
WHERE e.employee_id = r.employee_id
  AND r.rn > 1;
```

Alternative / notes:

```
-- First run the ranked CTE as SELECT to verify affected rows.
-- Use the true business key instead of name when appropriate.
```

Destructive operation: verify results and use suitable backup/transaction controls.

45. Troubleshoot a slow SQL query

Question: Troubleshoot a slow SQL query.

Solution:

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM orders
WHERE customer_id = 123;
```

Alternative / notes:

```
-- Review scans, estimated vs actual rows, joins, sort steps,
-- buffer reads, index use, and execution time.
-- Add indexes only when supported by query patterns and measurements.
```

Measure before/after. Indexes improve some reads but cost storage and write performance.

Part 6 — Backend, testing, data engineering, and full-stack

46. FastAPI endpoint with request validation

Question: Create a POST endpoint that validates a manufacturing event.

```
from datetime import datetime
from fastapi import FastAPI
from pydantic import BaseModel, Field

app = FastAPI()

class ProductionEvent(BaseModel):
    equipment_id: str = Field(min_length=1)
    count: int = Field(ge=0)
    event_time: datetime

@app.post("/events", status_code=201)
async def create_event(event: ProductionEvent):
    # Real implementation should persist the event.
    return {"accepted": True, "event": event}
```

Discuss validation, HTTP status codes, persistence, authentication, duplicate-event handling, observability, and tests.

47. Pagination and filtering in FastAPI

```
from fastapi import FastAPI, Query
app = FastAPI()
items = [{"id": i, "status": "ok" if i % 2 == 0 else "warning"}
         for i in range(1, 101)]

@app.get("/items")
def list_items(offset: int = Query(0, ge=0),
               limit: int = Query(20, ge=1, le=100),
               status: str | None = None):
    filtered = items if status is None else [x for x in items if x["status"] == status]
    return {"total": len(filtered), "items": filtered[offset:offset + limit]}
```

For production, paginate/filter in SQL rather than loading all records. Validate requested sort fields against an allowlist.

48. PostgreSQL query with SQLAlchemy

```
from sqlalchemy import create_engine, text

engine = create_engine(DATABASE_URL, pool_pre_ping=True)

def get_event(event_id: int):
    with engine.connect() as conn:
        row = conn.execute(
            text("SELECT id, equipment_id, count FROM production_events WHERE id = :id"),
            {"id": event_id},
        ).mappings().first()
    return dict(row) if row else None
```

Use parameterized queries, secure secret storage, transaction management, connection pooling, and an async driver if using async database calls.

49. Unit tests with pytest

```
# app.py
def total_units(events):
    return sum(event["count"] for event in events)

# test_app.py
from app import total_units

def test_total_units():
    assert total_units([{"count": 2}, {"count": 3}]) == 5

def test_empty_events():
    assert total_units([]) == 0
```

Add tests for invalid input, external API failures, database errors, duplicate messages, and expected error responses. Mock external dependencies at their boundaries.

50. Retry operation with exponential backoff

```
import time

def retry_call(func, attempts=3, base_delay=0.5,
               retryable=(TimeoutError, ConnectionError)):
    if attempts < 1:
        raise ValueError("attempts must be >= 1")
    for attempt in range(attempts):
        try:
            return func()
        except retryable:
            if attempt == attempts - 1:
                raise
            time.sleep(base_delay * (2 ** attempt))

print(retry_call(lambda: "success"))
```

Production improvements: jitter, capped delay, total timeout budget, honoring Retry-After, logging, and idempotency. Retry only transient failures.

51. Design a manufacturing telemetry pipeline

- Ingest equipment events through approved gateways/protocols such as OPC UA, MQTT, MES APIs, or historians.
- Define event IDs, equipment IDs, timestamps, units, schema versions, and validation rules.
- Use a durable broker when needed; handle duplicate and late events, backpressure, and dead-letter messages.
- Store operational data in an appropriate operational/time-series store and historical analytical data in a lakehouse such as Delta Lake when justified.
- Expose authenticated APIs and dashboards. Keep analytics separate from safety-critical control logic unless engineered and validated for that purpose.
- Add idempotent processing, replay/recovery, data-quality checks, metrics, logs, traces, alerts, and runbooks.
- Apply OT/IT security controls: segmentation, least privilege, managed secrets, audit logs, and security review.
- Test unit, contract, integration, load, failure-recovery, and data-quality scenarios.

52. Troubleshoot a slow API

- Reproduce and measure latency percentiles (p50/p95/p99), throughput, and error rate.
- Use traces/logs to identify time spent in the app, database, network, or downstream services.
- Check query plans, connection-pool saturation, CPU, memory, event-loop blocking, and dependency latency.
- Profile based on evidence, fix the measured bottleneck, then compare metrics and add regression tests.

53. JWT authentication and RBAC

- Validate token signature, issuer, audience, expiry, and permitted algorithms with a vetted library.
- Do not trust user-supplied role fields; authorize using verified identity and trusted claims or an authorization service.

- Enforce access controls on every protected endpoint/resource.
- Store secrets in a managed secret store, rotate keys, and log security-relevant events.
- Prefer established OAuth 2.0 / OpenID Connect flows over custom token cryptography.

54. React component fetching API data

```tsx
import { useEffect, useState } from "react";

type Event = { id: number; equipment_id: string; count: number };

export function EventsPanel() {
  const [events, setEvents] = useState<Event[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    const controller = new AbortController();
    async function load() {
      try {
        const response = await fetch("/api/events", { signal: controller.signal });
        if (!response.ok) throw new Error(`HTTP ${response.status}`);
        setEvents(await response.json());
      } catch (err) {
        if (!controller.signal.aborted) {
          setError(err instanceof Error ? err.message : "Unknown error");
        }
      } finally {
        if (!controller.signal.aborted) setLoading(false);
      }
    }
    load();
    return () => controller.abort();
  }, []);

  if (loading) return <p>Loading…</p>;
  if (error) return <p role="alert">{error}</p>;
  return <ul>{events.map(e =>
    <li key={e.id}>{e.equipment_id}: {e.count}</li>
  )}</ul>;
}
```

Discuss loading/error/empty states, accessibility, API contracts, cancellation, pagination, caching, testing, and authorization.

55. PySpark / Databricks data pipeline

```python
from pyspark.sql import functions as F

raw = spark.read.format("delta").load("/mnt/raw/production_events")
clean = (
    raw.filter(F.col("equipment_id").isNotNull())
       .filter(F.col("event_time").isNotNull())
       .withColumn("event_date", F.to_date("event_time"))
       .dropDuplicates(["event_id"])
)
(clean.write.format("delta")
      .mode("append")
      .partitionBy("event_date")
      .save("/mnt/curated/production_events"))
```

Discuss idempotent ingestion, schema evolution, partition strategy, small-file management, data-quality rules, incremental processing, orchestration, access controls, and late-arriving data. This is illustrative; production streaming pipelines need suitable checkpointing and recovery.

Suggested practice plan

Week	Focus	Practice
1	Strings, lists, dictionaries	3 problems daily; explain complexity
2	Hash maps, two pointers, sliding window	2 medium problems daily; test edge cases
3	Decorators, generators, exceptions, tests	Implement utilities and pytest tests
4	SQL joins, CTEs, window functions	2 SQL questions daily
5	FastAPI, PostgreSQL, auth, retries	Build a small REST API
6	React, data pipelines, system design	Explain an end-to-end manufacturing-data solution

Interview habit: Clarify requirements, state assumptions, cover empty and boundary inputs, write readable code, test at least one normal and one edge case, and explain time/space complexity.